Clairvoyance : Smart Glasses for the visually impaired in Indoor Environment

Gabriel Dupuis
MSc student
Institut Polytechnique de Paris - ENSTA
Paris, France
gabriel.dupuis@ensta-paris.fr

Amine Maazizi*
MSc student
Institut Polytechnique de Paris - ENSTA
Paris, France
amine.maazizi@ensta-paris.fr

Luana Garcia
Ayoub Boufous
Hussein Rammal
Ismail El Moufakir
Lara Polachini
Yasmin Van Den Broek
Louis Marchal

Abstract

People with severe visual impairment (*talk about old people*) face major challenges in their home, partly because existing assistive devices are expensive, intrusive, or require long adaptation times. The Clairvoyance project addresses this need by developing a low-cost system that provides real-time scene understanding through a combination of computer vision, speech processing, and a hybrid deterministic and language-model framework.

The approach integrates three components: a vision pipeline based on object detection (YOLO), depth estimation (MiDaS), and OCR; a deterministic module that handles predefined tasks such as reading text, describing the environment, and estimating distances; and a lightweight language-model component that manages free-form instructions when the user asks questions outside the predefined set. The system runs locally through a Raspberry Pi hardware module connected to an application that ensures audio feedback and voice interaction.

The prototype introduces a dual-framework design combining rule-based functions with an agentic module for improvisation. It is fully local, inexpensive to assemble with off-the-shelf components, and modular enough to support extensions such as calls, messages, and simple smartphone interaction.

Quantitative evaluations indicate an average end-to-end latency of about 1.3 seconds for vision-to-speech responses. Object detection and OCR achieve reliable performance for indoor use cases, depth estimation operates within acceptable error margins for household distances, and the total material cost remains under 100 euros excluding components already provided. These results show the feasibility of an accessible assistive device that supports essential daily tasks for visually impaired users.

CCS Concepts

• **Computing methodologies → Animation**.

*Equal contribution.

Craig-Boris '25, TBa, TAS, USA

ACM ISBN /25/01
https://doi.org/10.1145/3563045.3563055

Keywords

assistive technology; computer vision; speech interaction

ACM Reference Format:
Gabriel Dupuis, Amine Maazizi, Luana Garcia, Ayoub Boufous, Hussein Rammal, Ismail El Moufakir, Lara Polachini, Yasmin Van Den Broek, and Louis Marchal. 2025. Clairvoyance : Smart Glasses for the visually impaired in Indoor Environment. In *Craig-Boris '25: Craig's test event for Boris LaTeX Code 2 (Craig-Boris '25), January 1–5, 2025, TBa, TAS, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3563045.3563055

1 Introduction

Vision impairment is a widespread and growing public health concern. According to the World Health Organization, over 2.2 billion people globally live with some form of vision impairment or blindness, including about 1 billion cases that could have been prevented or addressed with existing care [World Health Organization 2019]. Such impairments severely affect individuals' ability to perform daily tasks independently, leading to reduced quality of life and increased reliance on caregivers. In particular, older adults are disproportionately impacted, as age-related eye conditions drive a rise in visual disability [Ehrlich et al. 2021]. In France alone, 1.3 million older individuals experience loss of autonomy at home, and nearly 2 million people live with partial or total blindness [INSEE 2024]. Many users abandon assistive devices due to steep learning curves or lack of integration with their living environments [Okonji et al. 2019]. These factors highlight the need for solutions that are affordable, intuitive, and adapted to the home.

Recent assistive systems for visually impaired users include both commercial and open-source efforts, yet most remain limited by high cost, restricted adaptability to domestic environments, or reliance on proprietary software. Products like OrCam MyEye and Envision Glasses integrate object recognition and OCR but cost upwards of $4,000 and often require internet connectivity for advanced features [Beers et al. 2025; Envision Technologies 2024]. Their interfaces—typically touchpad- or app-based—can be difficult to use for older adults with low technical literacy [Okonji et al. 2019]. In contrast, open-source projects such as Melencio's VIsION glasses demonstrate DIY alternatives based on components like Raspberry Pi and basic computer vision, but lack robustness, modularity, and integration with language reasoning [Melencio 2019]. At the technical level, monocular depth estimation models like MiDaS [Ranftl et al. 2021] enable affordable spatial perception compared to LiDAR or RGB-D sensors, which are expensive, energy-intensive, and poorly suited for transparent obstacles or indoor use [Benedetti 2023]. Lightweight OCR engines like PaddleOCR [PaddleOCR Contributors 2023] and object detectors such as YOLOv7 [Wang et al. 2022] offer high performance on edge devices. More recently, open-source large language models (LLMs), combined with orchestration frameworks like LangChain, support context-aware reasoning through agent-based workflows and retrieval-augmented generation (RAG) [Mialon et al. 2023]. However, existing systems rarely combine deterministic visual parsing with LLM-level improvisation in a unified, locally-executable, and affordable platform. Our work addresses this gap by integrating object detection, depth perception, OCR, and hybrid language processing into a single open-source wearable assistant designed specifically for unsupervised use in home environments.

Cost, usability, and integration remain significant gaps in the state of the art. Most AI-driven assistive glasses are prohibitively expensive, limiting accessibility in low-income settings and for uninsured individuals. There is also a usability gap: many visually impaired people, especially seniors, report difficulties in adopting new assistive technologies due to steep learning curves or insufficient accommodation of their needs. Some commercial glasses require handling a touchpad or smartphone app, which can be challenging for users with additional motor impairments or low technical literacy. Another shortcoming is the lack of environmental integration: existing devices function largely as standalone aids and do not interact with smart home systems or personalized location cues in the user's home. Furthermore, because most solutions are closed-source, users and rehabilitation specialists cannot easily customize or improve the software to better suit individual preferences.

This paper introduces an open-source assistive system that integrates object detection, depth estimation, OCR, intent classification, and fallback language generation within a hybrid visual-language pipeline. The system operates locally, requires no external connectivity, and runs on low-cost hardware. The remainder of the paper is organized as follows: Section 2 reviews related work. Section 3 details the system architecture and processing modules. Section 4 presents validation results. Section 5 discusses limitations and future work, and Section 6 concludes.

2 Methodology

2.1 Global Pipeline

Our product is separated into three parts: the client, that is hosted by the hardware of the user like the smartphone app or on a Rasberry Pi connected to a camera and microphone, and the server, that hosts all the models and computation necessary to process the text and image to give a correct output.

2.1.1 Client side. For the client, we have a Rasberry Pi version and a smartphone version. The client makes API call to Google text-to-speech to transform the user voice to a text and the text response to an audio. Between this, it also sends query transcription and camera information through an API to a remote server having the backend running.

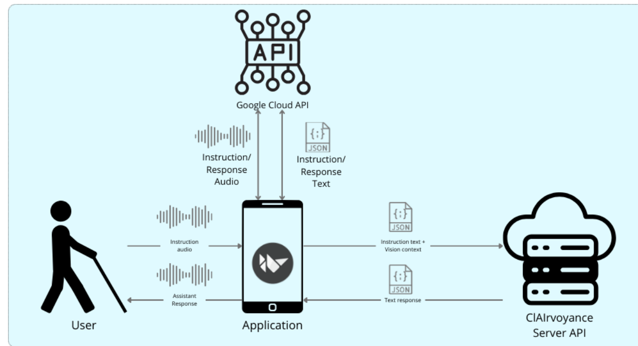

Figure 1: Pipeline on the client side

2.1.2 Server side. The server recceives the request, choose the right class of action attributed to the query of the user with a fine-tuned BERT, do a segmentation of the image with different vision models in parallel (YOLO, MiDaS, PaddleOCR) and computes the output to give according to the class and segmentation using another fine-tuned language model. If the class is miscellenaous, it uses LangChain and Qwen to do pre-defined agentic actions such as online search or information retrieval.

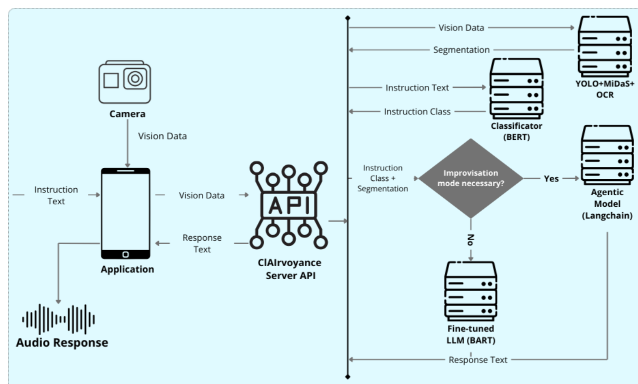

Figure 2: Pipeline on the server side

2.2 Vision and Scene Understanding

The vision subsystem of **Clairvoyance** integrates three interconnected components: **object detection**, **monocular depth estimation**, and **optical character recognition (OCR)**. These modules collectively produce a structured, multi-modal **vision hashmap** that serves as input for downstream reasoning and interaction.

2.2.1 Object Detection (YOLOv11). We use **YOLOv11-nano** for real-time detection of indoor objects. YOLOv11 introduces architectural innovations such as the **C3$_{k2}$ block**, **SPPF (Spatial Pyramid Pooling – Fast)**, and **C2PSA (Parallel Spatial Attention)**, which improve feature extraction efficiency without sacrificing accuracy [Khanam and Hussain 2024]. The model is trained on the **Microsoft COCO dataset** [Lin et al. 2014a], enabling it to detect a wide range of common objects in context. Detected objects are stored in a **hashmap** that includes the bounding box $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$, class label c, and confidence score p.

2.2.2 Monocular Depth Estimation (MiDaS). Depth is estimated from a single RGB image using the **MiDaS model** [Ranftl et al. 2021]. The pipeline consists of several stages:

Camera Calibration. We calibrate the camera with a checkerboard pattern to determine intrinsic parameters and distortion coefficients. Let $\{\mathbf{X}_i\}$ denote the known 3D checkerboard corners and $\{\mathbf{x}_i\}$ the corresponding 2D image points. The optimization problem solved is:

$$\min_{\mathbf{K},\mathbf{d},\{\mathbf{r},\mathbf{t}\}} \sum_i \left\|\mathbf{x}_i - \pi(\mathbf{K},\mathbf{d},\mathbf{r},\mathbf{t},\mathbf{X}_i)\right\|^2$$

where $\mathbf{K} \in \mathbb{R}^{3\times3}$ is the intrinsic camera matrix:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

with f_x, f_y as focal lengths and (c_x, c_y) the principal point. $\mathbf{d}$ contains distortion coefficients and π is the projection function. OpenCV's `calibrateCamera()` is used to estimate these parameters.

Depth Map Computation. Given an input frame I, the MiDaS depth $\hat{D}$ is computed via:

$$\hat{D} = \text{MiDaS}(T(I)), \quad \hat{D} \in \mathbb{R}^{H\times W}$$

where T is the preprocessing transform. Depth is resized to the original frame using bicubic interpolation: $D = \text{interp}(\hat{D}, (H, W))$.

Normalization and Conversion. Normalized depth is calculated:

$$D_{\text{norm}} = \frac{D - \min(D)}{\max(D) - \min(D)}$$

Real-world distances d_{real} in meters are computed using the focal length:

$$d_{\text{real}}(u,v) = \frac{f_x}{D_{\text{norm}}(u,v)}$$

An exponential moving average (EMA) is applied across frames to reduce noise:

$$d_t(u,v) = \alpha \cdot d_{\text{real},t}(u,v) + (1-\alpha)\cdot d_{t-1}(u,v)$$

Spatial Discretization. The depth map is partitioned into a $G_h \times G_w$ grid. For each bounding box b, the mean depth is:

$$\bar{d}_b = \frac{1}{|C(b)|}\sum_{(i,j)\in C(b)} g_{i,j}$$

Distance labels are then assigned using thresholding.

2.2.3 Text Recognition (PaddleOCR). Text recognition in **Clairvoyance** uses a two-stage pipeline. Text regions are first detected with the **EAST detector** [Zhou et al. 2017] and overlapping boxes are merged. Cropped regions are then processed by **PaddleOCR** [PaddleOCR Contributors 2023] with angle classification to handle rotated text. Recognized strings, bounding boxes, and confidence scores are stored in the **vision hashmap** under the `"text"` key, providing structured input for downstream reasoning.

2.2.4 Vision Output Format (JSON). The results from the vision subsystem are returned as a structured JSON object that consolidates object detection, depth estimation, and text recognition. Each frame produces a dictionary with at least two main keys: `objects` and `text`. The structure can be represented as:

```
{
  "objects": [
    {
      "class": "chair",
      "confidence": 0.87,
      "bbox": [50, 100, 200, 400],
      "center": [125, 250],
      "distance": 1.5,
      "depth_label": "within arm's reach"
    },
    {
      "class": "table",
      "confidence": 0.92,
      "bbox": [220, 80, 400, 350],
      "center": [310, 215],
      "distance": 2.8,
      "depth_label": "a bit farther"
    }
  ],
  "text": "Exit 12\nRoom 3"
}
```

2.3 Language and Reasoning

This component manages natural language understanding, intent classification, reasoning, and response generation. Planned, critical scenarios are handled deterministically using a fine-tuned **BERT** classifier [Devlin et al. 2019] and a **BART**-based response generator [Lewis et al. 2020], ensuring efficiency and avoiding hallucinations. Less critical, unplanned queries are handled by a fallback LLM orchestrated via `LangChain` [Chase 2022], trading additional compute and a small risk of hallucination for flexible, context-aware responses.

2.3.1 Intent Classifier. We fine-tune a **BERT-base-uncased** model for intent classification over six main classes (scene description, object distance, text reading, call, SMS, email) and a seventh class for unintended queries. The dataset is generated via an LLM-driven pipeline (`OllamaLLM`), producing 500 examples per class (totaling 3500 samples) with structured input-output pairs:

Question: [user query]
Answer: [concise answer]
Structured: [structured tuple]

The BERT classifier embeds the input, applies dropout, and passes it to a linear layer to produce logits:

$$\mathbf{logits} = \mathbf{W}\cdot \text{Dropout}(\text{BERT}_{CLS}(\text{input})) + \mathbf{b}, \quad \mathbf{logits} \in \mathbb{R}^C$$

2.3.2 Response Construction (BART-based). For intended queries (classes 0–5), responses are generated using a fine-tuned **BART**

encoder-decoder. Structured vision or context data is mapped to response templates:

$$R = \text{Template}(\text{StructuredData}, Q)$$

where R is the final text response, StructuredData is the extracted scene information, and Q is the user query. Cosine similarity may be applied for token-to-element mapping.

2.3.3 Fallback LLM. Class 6 queries (unintended/ambiguous) are handled by a fallback language model orchestrated via `LangChain` [Chase 2022]. Structured message sequences include system prompts, user profile, query, retrieved long-term memory, and short-term chat history. Tool calls (`<tool_call>`) are executed dynamically and integrated into subsequent reasoning iterations (typically up to two). The fallback model can access web search and external tools to supplement information, and final responses are post-processed to remove markers and extraneous tokens before delivery.

Speech Components. We use Google Text-to-Speech (TTS) [Graves 2012] and Speech-to-Text (STT) [Shen et al. 2018] API on the client-side. We also do regex recognition for the input and output to have reliable fallback answers in specific scenarion.

2.3.4 System architecture and components. The system is composed of a modular hardware platform and a lightweight software stack designed for local or hybrid execution. The core hardware includes a Raspberry Pi 4 Model B, a wide-angle USB camera, bone-conduction audio output, and an elastic mounting band, forming a head-worn device. The software runs a synchronous API on the Pi that orchestrates perception modules (YOLOv11, MiDaS, PaddleOCR), a lightweight instruction classifier, and fallback LLM reasoning. User interactions occur via voice only: the system captures speech through a local or smartphone microphone, transcribes it via a cloud-based STT API (Google Cloud or Whisper), and processes the instruction using an intent classifier. Processed data are structured as JSON packets containing vision elements (object tags, bounding boxes, depth classes, OCR output), which are either answered deterministically or routed to a fine-tuned LLM agent pipeline. The architecture supports both edge-only deployment (with all reasoning on the Pi) and hybrid modes where the LLM runs on a remote server accessible via LAN. This design prioritizes offline functionality and privacy while allowing higher-level language capabilities when connected.

3 Results

3.1 Delay of the pipeline

The pipeline achieves an average end-to-end latency of 1.36 seconds across all query types. The primary bottleneck is text-to-speech synthesis at 1.08s (range 0.27–5.40s), while visual processing components remain efficient: object detection 42ms, depth estimation 16ms, and OCR 91ms. Request classification adds 124ms overhead, while image encoding and decoding operations are negligible at 3–4ms each.

Table 1: Mean pipeline execution times (20 requests)

Component	Time (s)
Total Pipeline	1.364
Image Decode	0.003
Request Classification	0.124
Object Detection	0.042
Depth Estimation	0.016
Text Recognition (OCR)	0.091
TTS Synthesis	1.077
Image Encode	0.004

3.2 Evaluation of the BERT Intent Classifier

The fine-tuned BERT intent classifier demonstrates solid performance for routing user queries within the Clairvoyance pipeline. On a held-out synthetic evaluation set, the model achieves an overall accuracy of 0.759 and a weighted F_1 score of 0.748, with a high macro AUC-ROC of 0.977, indicating strong separability between classes. Performance varies across intents: operational commands such as *Call* and *SMS* achieve the highest recall (1.00 and 0.90 respectively), reflecting the consistency of their linguistic patterns. Vision-related intents such as *Objects* and *Text* also show strong precision (0.89 and 0.93), suggesting reliable detection when triggered. Lower recall is observed for *Distance* and *Email*, likely due to greater semantic variability in phrasing within the synthetic dataset. Importantly, inference latency remains low (mean 58.5 ms on CPU), ensuring that intent classification does not become a bottleneck in the real-time pipeline. Overall, these results indicate that BERT provides an effective and computationally efficient routing mechanism for hybrid deterministic–LLM interaction.

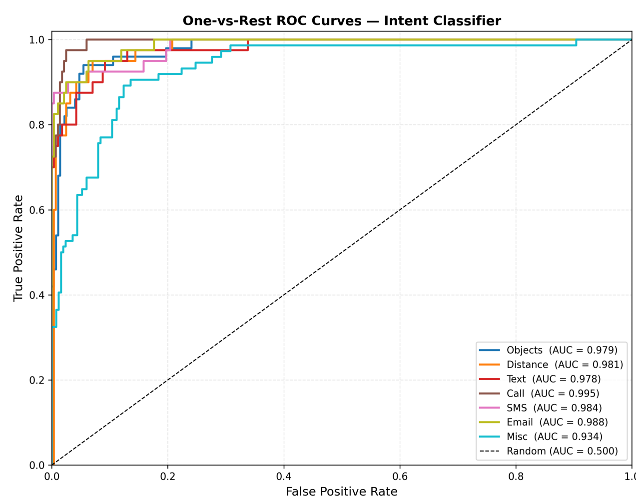

Figure 3: ROC Curves associated to fine-tuned BERT per class

3.3 End-to-End Response Validation

To assess the quality of the Clairvoyance pipeline in realistic conditions, we built a synthetic benchmark derived from the COCO

Table 2: Global evaluation metrics for the BERT intent classifier (7-class, synthetic held-out set).

Metric	Value
Accuracy	0.7593
Macro Precision	0.8343
Macro Recall	0.7401
Macro F_1	0.7454
Weighted F_1	0.7479
AUC-ROC (macro OvR)	0.9769
Mean latency (CPU)	58.5 ms
Median latency (CPU)	55.5 ms

Table 3: Per-class evaluation metrics of the BERT intent classifier on the held-out synthetic test set.

Class	Support	Precision	Recall	F1	Accuracy
Objects	50	0.8889	0.8000	0.8421	0.8000
Distance	40	0.9524	0.5000	0.6557	0.5000
Text	40	0.9333	0.7000	0.8000	0.7000
Call	40	0.6897	1.0000	0.8163	1.0000
SMS	40	0.7500	0.9000	0.8182	0.9000
Email	40	1.0000	0.3750	0.5455	0.3750
Misc	74	0.6262	0.9054	0.7403	0.9054

val2017 dataset [Lin et al. 2014b]. For each of the 99 samples, a natural-language query was paired with the corresponding image and submitted to the running pipeline via the `/process_remote` REST endpoint. A human evaluator then labelled each pipeline response as *correct* or *incorrect* by comparing the answer to the actual image content, without access to ground-truth COCO annotations.

Results are shown in Table 4. The pipeline achieves an overall accuracy of **69.7%**. Performance varies significantly across the three task classes. Object detection queries (class 0) yield the highest accuracy (**86.1%**): YOLO11n reliably enumerates the salient objects present in the scene, and the template-based language module constructs grammatically consistent answers. Distance estimation (class 1) reaches **63.0%**: the monocular depth estimation provides a coarse but often plausible ordering, yet the pipeline systematically reports the highest-confidence detection rather than the object explicitly named in the query, leading to mismatches when that object is not the most prominent one. Text recognition (class 2) is the weakest module at **52.9%**: the EAST detector fails on certain aspect ratios, causing an OpenCV `ConcatLayer` error that silently returns an empty result, and PaddleOCR returns `None` for images with no clearly legible text.

Table 4: Human validation of pipeline responses on a 99-sample COCO-derived benchmark (80 class-0/1 queries, 17 class-2 queries, 2 class-1 queries flagged ambiguous).

Task (class)	Samples	Correct	Accuracy (%)
Object Detection (0)	36	31	86.1
Distance Estimation (1)	46	29	63.0
Text Recognition (2)	17	9	52.9
Overall	**99**	**69**	**69.7**

4 Discussion

4.1 Pipeline Efficiency

The Clairvoyance pipeline demonstrates that a multimodal assistive system combining computer vision, intent classification, and speech interaction can operate within practical real-time constraints. The measured end-to-end latency of 1.36 s is dominated by text-to-speech synthesis, while perception modules remain lightweight (YOLO detection 42 ms, MiDaS depth estimation 16 ms, OCR 91 ms). These results indicate that modern compact models enable real-time scene understanding without specialized hardware. Compared with commercial assistive glasses, which often rely on proprietary hardware or cloud services, our system emphasizes affordability and modular open-source components while maintaining acceptable responsiveness for indoor use.

4.2 BERT Intent Classifier

The fine-tuned BERT classifier provides an efficient routing mechanism for user queries, achieving 0.76 accuracy and a weighted F_1 score of 0.75 with sub-60 ms inference latency on CPU. Performance is strong for operational commands such as calls or SMS, while lower recall for intents such as *Distance* or *Email* reflects greater linguistic variability. Some proximity queries (e.g. "Is the truck in front of me?") are misclassified as miscellaneous, indicating insufficient coverage of relational phrasing in the training data. Expanding the dataset with additional paraphrases and directional expressions should improve routing reliability.

4.3 System Limitations and Future Improvements

Several limitations remain in the current pipeline. The response generator, based on a fine-tuned language model and templates, produces reliable but sometimes limited responses; larger or more flexible models could improve expressiveness. The OCR module (PaddleOCR) performs well on clear text but is slow and struggles with distant or blurred text; video-based OCR approaches or more robust detectors such as DBNet could improve performance. While YOLO detects objects reliably, the system currently captures limited relational information between them; incorporating spatial reasoning or scene graphs would enable richer descriptions. The agentic reasoning component is also still limited and could be extended with stronger planning and tool orchestration. Finally, evaluation was conducted on COCO images, which mainly depict outdoor scenes, whereas the system is designed for indoor environments.

Future work should therefore evaluate the pipeline on first-person indoor datasets better aligned with the intended use case.

5 Conclusion and Future Work

This work presented *Clairvoyance*, an open-source assistive system that combines computer vision, intent classification, and language generation to support visually impaired users in indoor environments. The proposed pipeline integrates object detection, monocular depth estimation, OCR, and a hybrid language framework based on a fine-tuned BERT classifier and deterministic response generation. Experimental results show that the system can operate with an average end-to-end latency of approximately 1.3 seconds while maintaining reliable performance for key tasks such as object identification and scene description. These results demonstrate the feasibility of building a low-cost and modular assistive device using commodity hardware and open-source models.

Several directions remain for future work. First, perception modules could be improved by incorporating more robust text detection and video-based OCR methods, as well as richer spatial reasoning between detected objects. Second, the language layer could be extended with larger or more flexible models to generate more natural responses and support more complex queries. Third, the agentic reasoning component could be expanded to allow multi-step planning and more advanced tool usage. Finally, future evaluations should be conducted on datasets and recordings specifically collected in indoor first-person environments, which better reflect the intended real-world usage of the system.

References

Kevin Beers, Samantha Li, and Thao Nguyen. 2025. AI in assistive eyewear: Trends and challenges in smart glasses for visual support. *Journal of Wearable Computing* 18, 1 (2025), 14–27. Forthcoming.

Luca Benedetti. 2023. Smart Glasses: Limitations of LiDAR-based indoor depth detection. Technical Report. https://www.luca-benedetti.com/lidar-glasses-depth

Harrison Chase. 2022. *LangChain*. https://github.com/langchain-ai/langchain

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2019). https://arxiv.org/abs/1810.04805

Joshua R. Ehrlich, Brian C. Stagg, and Carol A. Andrews. 2021. Vision impairment and subjective cognitive decline among adults aged 45 years. *Preventive Medicine Reports* 21 (2021), 101314. doi:10.1016/j.pmedr.2020.101314

Envision Technologies. 2024. Envision Glasses: AI-powered smartglasses for people that are blind or have low vision. https://www.letsenvision.com

Alex Graves. 2012. Sequence Transduction with Recurrent Neural Networks. *arXiv preprint arXiv:1211.3711* (2012).

INSEE. 2024. Les personnes âgées en perte d'autonomie en 2024. https://www.insee.fr/fr/statistiques/7716002

Rahima Khanam and Muhammad Hussain. 2024. YOLOv11: An Overview of the Key Architectural Enhancements. *arXiv preprint arXiv:2410.17725* (2024).

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)* (2020), 7871–7880. https://arxiv.org/abs/1910.13461

Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. 2014a. Microsoft COCO: Common Objects in Context. *arXiv preprint arXiv:1405.0312* (2014). https://doi.org/10.48550/arXiv.1405.0312

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. 2014b. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision (ECCV)*. Springer, 740–755.

Michael Melencio. 2019. Open source smart glasses for the visually impaired. GitHub repository. https://github.com/melencio/vision-glasses

Guillaume Mialon, Vladislav Lialin, Thomas Scialom, Guillaume Staerman, Felipe Llinares-López, Hervé Jegou, Antoine Tixier, and Alexandre Lacoste. 2023. Augmented Language Models: A Survey. arXiv preprint. arXiv:arXiv:2302.07842 https://arxiv.org/abs/2302.07842

Edith F. Okonji, Garth Simons, Rizwan Majeed, and Cheryl Sloan. 2019. Barriers to adoption of assistive technologies for older adults with visual impairment. *Disability and Rehabilitation: Assistive Technology* 14, 6 (2019), 568–576. doi:10.1080/17483107.2018.1526733

PaddleOCR Contributors. 2023. PaddleOCR: Multilingual and lightweight OCR system. GitHub repository. https://github.com/PaddlePaddle/PaddleOCR

Rene Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. 2021. Vision transformers for dense prediction. *arXiv preprint* (2021). arXiv:arXiv:2103.13413 https://arxiv.org/abs/2103.13413

Jonathan Shen, Ruoming Pang, Ron J. Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, RJ Skerry-Ryan, et al. 2018. Natural TTS Synthesis by Conditioning WaveNet on Mel Spectrogram Predictions. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 4779–4783.

Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. 2022. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. arXiv preprint. arXiv:arXiv:2207.02696 https://arxiv.org/abs/2207.02696

World Health Organization. 2019. World report on vision. https://www.who.int/publications/i/item/9789241516570

Xinyu Zhou, Cong Yao, He Wen, Yuzhi Wang, Shuchang Zhou, and Liang He. 2017. EAST: An Efficient and Accurate Scene Text Detector. In *CVPR*. 5551–5560.